Neovim Pocket Guide

Your config is a deliberately small setup: a fast place to read and fix what agents wrote. It has five leader keys, a git UI, and language servers for TypeScript, Python, Swift, Lua, the web, shell, YAML and Markdown.

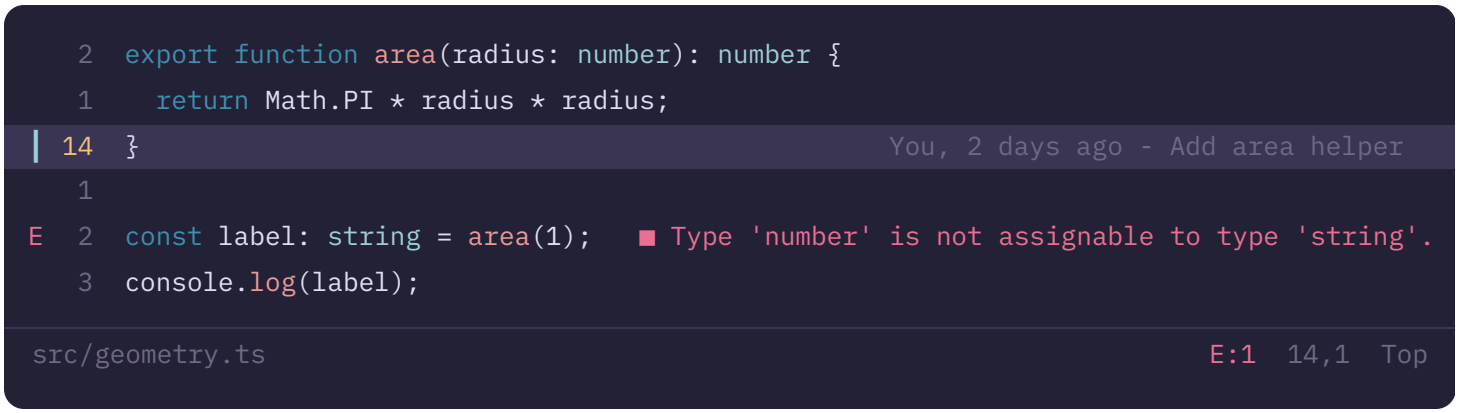
`Space` is the leader key. Press it and pause: a menu lists every leader key, so you don't need to memorize them.

Everyday keys · Code intelligence · Finding things · Files · Git · Watch out · Maintenance

: Everyday keys

These cover most of a session. Everything else is built into Neovim.

`Space` `f`	Find a file by name
`Space` `s`	Search text across the project
`Space` `b`	Switch between open files
`Space` `e`	Open the current file's folder in Oil, where you edit files like text
`Space` `g`	Open Neogit: status, stage, commit, push
`g` `d`	Go to the definition of the word under the cursor Files with a language server only. `Ctrl-o` jumps back.
`K`	Show the type and docs for the word under the cursor
`Esc`	Save the file (in normal mode) Handy, with some side effects. See Watch out.
`Ctrl-a`	Select the whole file
`p`	Paste over a selection without losing your clipboard



Sign column, far left: E error, W warning, H hint, and a bar for lines changed since the last commit.

Line numbers: the cursor line shows its real number, the others count up and down from it, so `2` `j` lands two lines down.

End of line: errors and warnings appear in color. Git blame shows on the cursor line only.

Status line: problem counts for the file, then line and column.

: Code intelligence

Language servers read your project the way VS Code does. They start automatically when you open a matching file.

LANGUAGE	SERVER	NOTES
TypeScript, JavaScript, React	`vtsls`	Same engine as VS Code. Uses each project's own TypeScript version.
Python	`pyright`	Uses the project's `.venv` when there is one, so installed packages resolve.
Swift	`sourcekit-lsp`	Ships with Xcode. Xcode projects need a one-time setup, see Maintenance.
Lua	`lua_ls`	Knows the Neovim API and Snacks, for editing this config.
HTML	`html`	Tag and attribute completion, and docs on hover. Same server as VS Code.
CSS	`cssls`	Property completion, and flags mistakes like a misspelled property.

LANGUAGE	SERVER	NOTES
JSON	`jsonls`	Syntax errors, plus validation for files that name a `$schema`.
Tailwind CSS	`tailwindcss`	Class completion and hover in projects that use Tailwind. Idle everywhere else.
Bash, shell	`bashls`	Problems come from `shellcheck`, which the install script adds too.
YAML	`yamlls`	Picks the right schema for files like GitHub workflows and docker-compose.
Markdown	`marksman`	Completes and follows links and headings, and flags links to files that don't exist.

MOVING AROUND CODE

`g` `d`	Go to definition With several results, a picker opens.
`Ctrl-o` / `Ctrl-i`	Jump back / forward through where you've been
`K`	Type and docs. Press `K` again to move into the popup and scroll it.
`g` `r` `r`	List every reference
`g` `r` `i`	Go to implementation
`g` `r` `t`	Go to the type's definition
`g` `O`	Outline of every symbol in the file

CHANGING CODE

`g` `r` `n`	Rename a symbol everywhere it's used

`g` `r` `a`	Code actions: quick fixes, add missing import, organize imports Also works on a visual selection.
`Ctrl-s`	In insert mode: show the parameters of the function you're calling

COMPLETION

A menu opens as you type in code files. It lists what the language server suggests first, then words from your open files. Keep typing to ignore it. In Markdown it only opens for links and headings, so prose stays quiet.

`Ctrl-n` / `Ctrl-p`	Move down / up the list. The highlighted item is filled in as you move.
`Ctrl-y`	Accept the item Accepting a function from another file also adds its import line at the top.
`Ctrl-e`	Close the menu and put back what you typed
`Enter`	Always starts a new line. It never accepts a suggestion.

ERRORS AND WARNINGS

`]` `d` / `[` `d`	Next / previous problem in the file
`]` `D` / `[` `D`	Last / first problem in the file
`Ctrl-w` `d`	Show the full message under the cursor in a popup

`:lua Snacks.picker.diagnostics()` lists every problem in your open files, in a picker.

Hints, such as an unused variable, don't get a message at the end of the line. The variable is dimmed and an H shows in the sign column. Language servers only check files you have open. To check a whole TypeScript project, run `npx tsc --noEmit` in it.

: Finding things

Inside any picker (`Space` `f`, `s`, `b`), you type to filter and these keys act on the results.

`Ctrl-j` / `Ctrl-k`	Move down / up
`Enter`	Open
`Ctrl-v` / `Ctrl-s`	Open in a vertical / horizontal split
`Tab`	Select several results
`Ctrl-q`	Send the results to the quickfix list (Neovim's list of locations)
`Alt-h` / `Alt-i`	Include hidden / gitignored files
`Alt-p`	Show or hide the preview. `Ctrl-f` and `Ctrl-b` scroll it.
`Ctrl-g`	In text search: stop searching and filter the current matches instead
`Esc` `Esc` / `Ctrl-c`	Close the picker The first `Esc` only stops typing.
`?`	Show every picker key

FIND AND REPLACE ACROSS THE PROJECT

1. `Space` `s` and type `oldName`
2. `Ctrl-q` sends every match to the quickfix list
3. Run `:cdo s/oldName/newName/g | update` to replace in each match and save
4. `]` `q` and `[` `q` step through the list to review

For renaming a variable or function in code, `g` `r` `n` is safer: it only changes real uses of that symbol.

MORE PICKERS

These have no key yet. Run them as commands, or ask to have any of them mapped.

`:lua Snacks.picker.resume()`	Reopen the last picker exactly where you left it
`:lua Snacks.picker.recent()`	Files you opened recently
`:lua Snacks.picker.grep_word()`	Search for the word under the cursor
`:lua Snacks.picker.git_status()`	Files with uncommitted changes
`:lua Snacks.picker.keymaps()`	Every key mapping, searchable
`:lua Snacks.picker()`	A picker of all pickers

: Files with Oil

`Space` `e` shows the current file's folder as a text buffer. Edit it like text, then save to apply.

Rename	Edit the name
Create	Add a new line. End it with `/` for a folder.
Delete	`d` `d` on the line
Move	`d` `d` here, then `p` in another Oil folder
Apply	Save. Oil lists the changes and asks you to confirm.

KEYS INSIDE OIL

`-`	Up to the parent folder
`Enter`	Open the file or folder
`Ctrl-p`	Preview the file
`Ctrl-s`	Open in a vertical split
`_`	Jump to the folder Neovim was started in

`Ctrl-c`	Close Oil
`g` `?`	Show every Oil key

: Git

`Space` `g` opens Neogit's status view. Keys act on the file or hunk under the cursor.

`Tab`	Expand or collapse a file's diff
`s` / `u`	Stage / unstage `S` and `U` do it for everything.
`x`	Discard the change
`c` `c`	Commit
`P` / `p` / `f`	Push / pull / fetch menus
`b` / `l` / `Z`	Branch / log / stash menus
`d`	Diff menu, opens the change in Diffview
`?`	Every menu and key
`q`	Close Neogit

WRITING THE COMMIT MESSAGE

`Ctrl-c` `Ctrl-c`	Commit
`Ctrl-c` `Ctrl-k`	Cancel the commit
`Alt-p` / `Alt-n`	Bring back earlier commit messages

GITSIGNS

Gitsigns shows the bar in the sign column and the blame on the cursor line. It has no keys, so use commands:

`:Gitsigns preview_hunk`	Show what changed in this hunk
`:Gitsigns nav_hunk next`	Jump to the next changed hunk
`:Gitsigns reset_hunk`	Undo the changes in this hunk
`:Gitsigns stage_hunk`	Stage just this hunk
`:Gitsigns blame`	Blame for the whole file, in a side panel

`lazygit` is also installed if you'd rather use git from a terminal UI.

: Watch out

Things in the config that can surprise you. None of these were changed. Each one has a suggested fix if you want it.

RISK `Esc` **always runs** `:w`

- In a new unsaved buffer or in `:help`, it shows an error (`E32: No file name`, `E45: 'readonly' option is set`).
- In Oil, it applies your pending file changes (after a confirmation).
- In a Neogit commit message, once `Esc` has saved, `q` commits right away instead of asking. Cancel with `Ctrl-c` `Ctrl-k`.
- Most important with agents: `:w` writes even when you changed nothing. If an agent edited the file after you opened it, Neovim asks whether to overwrite, and one `y` wipes the agent's work. `:update` only writes when you made a change.

Suggested fix, in `lua/keys.lua`:

```
vim.keymap.set('n', '<Esc>', function()
  vim.cmd.nohlsearch()  -- also clear search highlight
  if vim.bo.buftype == '' and vim.api.nvim_buf_get_name(0) ~= '' then
    vim.cmd('silent update')
  end
end, { desc = 'Save' })
```

With this version, `Esc` no longer applies Oil changes, so use `:w` there.

MINOR **Reload a file an agent changed**

Run `:e` to reload the current file from disk, or `:checktime` to check every open file.

MINOR `g` `d` **needs a language server**

It's wired to the language server, so in files without one, such as plain text, it finds nothing. In Markdown it follows links. It replaces Neovim's own `g` `d`, which jumps to where a name is first declared in the file.

MINOR `Ctrl-a` **replaces "add 1 to the number"**

Neovim normally uses `Ctrl-a` to increase the number under the cursor (`Ctrl-x` still decreases it). If you only select all to copy the file, `:%y` does it in one step.

MINOR **The paste mapping isn't needed**

Capital `P` over a selection already pastes without overwriting your clipboard. The mapping does no harm.

MINOR **The mouse is off**

`mouse = ''` in `lua/vim_config.lua` turns it off. `mouse = 'a'` turns on scrolling and clicking in WezTerm.

: Maintenance

SET UP SWIFT FOR ANOTHER XCODE PROJECT

1. In the project folder: `xcode-build-server config -project Name.xcodeproj -scheme Scheme` (`xcodebuild -list` shows the schemes)
2. Build once, in Xcode or with `xcodebuild -scheme Scheme -destination 'generic/platform=iOS Simulator' build`
3. Keep the generated `buildServer.json` out of git. It holds paths from this machine. `echo buildServer.json >> .git/info/exclude` ignores it without touching `.gitignore`.

After adding files or targets, build again so the server picks them up.

PYTHON VIRTUAL ENVIRONMENTS

pyright uses `.venv` in the folder that holds `pyproject.toml`. Without one, third-party imports show as unresolved. `uv venv` creates it and `uv sync` installs the project's packages into it. Run `:lsp restart` afterwards.

ADD ANOTHER LANGUAGE

1. Find the server's name: each file in `~/.local/share/nvim/lazy/nvim-lspconfig/lsp/` is one server, and its header says how to install it
2. Add its install command to `scripts/17-language-servers.sh` and run `./install.sh 17`
3. Add the name to `vim.lsp.enable({ ... })` in `lua/plugins/lsp.lua`

UPDATING

`:Lazy update`	Update plugins. Rewrites `lazy-lock.json`, which pins versions for a rebuild.

`./install.sh 17`	Install or update every language server
`./install.sh 16`	Move Node to the newest LTS
`brew upgrade`	Update the brew tools

WHEN SOMETHING'S OFF

`:checkhealth vim.lsp`	Which servers are running for this file, and their settings
`:lsp restart`	Restart the servers for this file, e.g. after installing packages
`~/.local/state/nvim/lsp.log`	Server errors. `Command setContext not found` lines from vtsls are harmless.
`:checkhealth snacks`	If a picker misbehaves

Generated from your config in `~/workbench`. Every key in this guide was checked against the installed plugins and Neovim 0.12.6.